



Projet Python M2

Description du projet

Nathalie Qiu

I. Sujet et contexte

Le projet porte sur l'**analyse et le traitement de données de risque de crédit bancaire**, en lien direct avec les problématiques rencontrées lors du stage.

Dans un contexte bancaire, les équipes de gestion des risques doivent analyser de larges volumes de données clients afin d'évaluer l'exposition au risque de crédit, de produire des indicateurs synthétiques et d'alimenter des reportings internes.

L'objectif de ce projet est de **reproduire, de manière pédagogique, un processus réaliste de traitement de données bancaires**, allant de la constitution d'une base de données clients jusqu'à la production d'un reporting final automatisé.

Le projet se concentre sur des notions clés de la gestion du risque de crédit, telles que :

- l'exposition au défaut (EAD),
- la probabilité de défaut (PD),
- la perte en cas de défaut (LGD),
- la perte attendue (Expected Loss),
- l'identification des clients nécessitant un suivi renforcé (Watchlist),
- et une analyse simplifiée de conformité aux seuils prudentiels (proxy Bâle III).

II. Sources de données

Aucune base de données réelle n'était disponible pour ce projet. Pour cette raison, un **premier script Python de génération de données** a été développé afin de créer une base de données cohérente et réaliste.

Ce script génère un fichier Excel structuré contenant **trois jeux de données distincts** :

1. **Clients**
 - Identifiant client
 - Pays
 - Secteur d'activité
 - Rating interne
2. **Exposures (expositions de crédit)**
 - Identifiant client
 - Montant du prêt (principal)
 - EAD (exposition au défaut)
 - PD (probabilité de défaut)
 - LGD (perte en cas de défaut)
3. **Collaterals (collatéraux)**
 - Identifiant client
 - Valeur du collatéral

Cette approche permet de simuler une **base de données relationnelle** proche de celles utilisées en environnement bancaire, tout en respectant un cadre strictement pédagogique.

Voici le code : (Voir annexe aussi)

```
1 import numpy as np
2 import pandas as pd
3 from pathlib import Path
4
5 # PARAMETRES
6
7 N_CLIENTS = 10000
8
9 # Fichier Final
10 OUTPUT_FILE = Path("/Users/nathalyqiu/Desktop/data/raw/bank_database.xlsx")
11
12 # (optionnel)
13 SEED = 42
14
15 # GENERATION DES DONNEES
16
17 rng = np.random.default_rng(SEED)
18
19 countries = ["France", "Germany", "Italy", "Spain", "Belgium", "Netherlands"]
20 sectors = ["Industry", "Services", "Retail", "Energy", "Tech", "RealEstate"]
21 ratings = ["AAA", "AA", "A", "BBB", "BB", "B"]
22
23 # ----- TABLE CLIENTS -----
24 clients = pd.DataFrame()
25 clients["client_id"] = [f"C{i:05d}" for i in range(1, N_CLIENTS + 1)]
26 clients["country"] = rng.choice(countries, size=N_CLIENTS)
27 clients["sector"] = rng.choice(sectors, size=N_CLIENTS)
28 clients["rating"] = rng.choice(ratings, size=N_CLIENTS, p=[0.03, 0.07, 0.20, 0.35, 0.25, 0.10])
29
30 # ----- TABLE EXPOSURES -----
31 # EAD : distribution réaliste très étalée
32 ead = rng.lognormal(mean=12.0, sigma=0.8, size=N_CLIENTS)
33 ead = np.clip(ead, 10000, 15000000)
34
35 # Montant du prêt :
36 ratio_loan = rng.uniform(0.6, 1.0, size=N_CLIENTS)
37 loan_amount = ead * ratio_loan
38
39 # PD et LGD
40 pd_values = rng.beta(a=1.5, b=50, size=N_CLIENTS) * 0.25
41 pd_values = np.clip(pd_values, 0.0001, 0.20)
42
43 lgd_values = rng.beta(a=2.0, b=2.5, size=N_CLIENTS)
44 lgd_values = np.clip(lgd_values, 0.10, 0.90)
45
46 exposures = pd.DataFrame()
47 exposures["client_id"] = clients["client_id"]
48 exposures["loan_amount"] = loan_amount.round(2)
49 exposures["ead"] = ead.round(2)
50 exposures["pd"] = pd_values.round(6)
51 exposures["lgd"] = lgd_values.round(4)
52
53 # ----- TABLE COLLATERALS -----
54 n_lines = rng.integers(0, 4, size=N_CLIENTS) # 0,1,2,3
55 client_ids = np.repeat(clients["client_id"].values, n_lines)
56
57 collateral_value = rng.lognormal(mean=10.5, sigma=0.7, size=len(client_ids))
58 collateral_value = np.clip(collateral_value, 5000, 5000000)
59
60 collaterals = pd.DataFrame()
61 collaterals["client_id"] = client_ids
62 collaterals["collateral_value"] = collateral_value.round(2)
63
64 # EXPORT EXCEL
65
66 OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)
67
68 with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:
69     clients.to_excel(writer, sheet_name="clients", index=False)
70     exposures.to_excel(writer, sheet_name="exposures", index=False)
71     collaterals.to_excel(writer, sheet_name="collaterals", index=False)
72
73 print("Fichier Excel généré :", OUTPUT_FILE)
74 print("Clients :", len(clients))
75 print("Exposures :", len(exposures))
76 print("Collaterals lines :", len(collaterals))
77
```

III. Principales transformations de données

Un second script Python est dédié au **traitement et à l'analyse des données** générées. Il repose exclusivement sur les notions vues durant le cours.

Les principales étapes de traitement sont les suivantes :

1. Nettoyage des données

- Suppression des valeurs manquantes sur les champs critiques.
- Conversion des variables numériques (montants, ratios).
- Vérification de la cohérence des données (PD et LGD entre 0 et 1, EAD supérieur au montant du prêt).

2. Jointures de données

- **Jointure** entre les tables *exposures* et *clients* (équivalent RECHERCHEV Excel).
- Gestion d'un cas **one-to-many** pour les collatéraux, avec agrégation préalable des montants par client avant jointure.

3. Création d'indicateurs de risque

- Calcul de la **perte attendue** : $Expected\ Loss = EAD \times PD \times LGD$.
- Identification des clients en **Watchlist**, définie par une PD supérieure à 3 %.
- Calcul du taux de couverture par les collatéraux.

4. Analyse prudentielle

Une analyse simplifiée de conformité aux seuils de fonds propres est réalisée :

- calcul d'un ratio CET1 proxy,
- comparaison à plusieurs seuils réglementaires (4,5 %, 8 %, 10,5 %),
- production d'un tableau synthétique indiquant le nombre et le pourcentage d'expositions conformes.

Ces calculs reposent sur des hypothèses simplifiées et n'ont pas vocation à reproduire un calcul réglementaire exact.

5. Agrégations et visualisations

- Agrégation des données par pays et par secteur d'activité (groupby, sum, mean).
- Production de graphiques simples (distribution des PD, exposition par pays, taux de Watchlist par secteur).

Voici le code : (Voir annexe aussi)

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5  from pathlib import Path
6  from openpyxl.drawing.image import Image as XLImage
7
8
9
10 # PARAMETRES
11
12 # Fichier source
13 INPUT_EXCEL = Path("/Users/nathalyqiu/Desktop/data/raw/bank_database.xlsx")
14
15 # Fichier final
16 OUTPUT_EXCEL = Path("/Users/nathalyqiu/Desktop/data/raw/bank_risk_report.xlsx")
17
18 PD_WATCHLIST_THRESHOLD = 0.03 # Seuil = Zone de vigilance utilisé par les banques
19
20 # Basel III (proxy)
21 BASEL_45 = 0.045 # Règlementation = CET1 min
22 BASEL_8 = 0.08 # Tier 1 + Tier 2 = Capital total min
23 BASEL_105 = 0.105 # CET1 + Buffer
24
25 RW_PROXY = 1.0 # Cas Standard
26 CET1_RATE_PROXY = 0.12 # Banque prudente donc sup à 10,5%
27
28 # Dossier pour images
29 FIG_DIR = Path("/Users/nathalyqiu/Desktop/data/raw/figures_tmp")
30
31
32 # 1) DONNÉES
33
34 if not INPUT_EXCEL.exists():
35     raise FileNotFoundError(f"Fichier introuvable : {INPUT_EXCEL}")
36
37 clients = pd.read_excel(INPUT_EXCEL, sheet_name="clients")
38 exposures = pd.read_excel(INPUT_EXCEL, sheet_name="exposures")
39 collaterals = pd.read_excel(INPUT_EXCEL, sheet_name="collaterals")
40
41
42
43 # 2) NETTOYAGE
44
45 clients = clients.dropna(subset=["client_id"]).copy()
46 clients["client_id"] = clients["client_id"].astype(str).str.strip()
47 clients["country"] = clients["country"].astype(str).str.strip()
48 clients["sector"] = clients["sector"].astype(str).str.strip()
49 clients["rating"] = clients["rating"].astype(str).str.strip().str.upper()
50
51 exposures = exposures.dropna(subset=["client_id", "loan_amount", "ead", "pd", "lgd"]).copy()
52 exposures["client_id"] = exposures["client_id"].astype(str).str.strip()

```

```

53
54 exposures["loan_amount"] = pd.to_numeric(exposures["loan_amount"], errors="coerce")
55 exposures["ead"] = pd.to_numeric(exposures["ead"], errors="coerce")
56 exposures["pd"] = pd.to_numeric(exposures["pd"], errors="coerce")
57 exposures["lgd"] = pd.to_numeric(exposures["lgd"], errors="coerce")
58 exposures = exposures.dropna(subset=["loan_amount", "ead", "pd", "lgd"])
59
60 exposures = exposures[(exposures["loan_amount"] > 0) & (exposures["ead"] > 0)]
61 exposures = exposures[exposures["pd"].between(0, 1)]
62 exposures = exposures[exposures["lgd"].between(0, 1)]
63 exposures = exposures[exposures["loan_amount"] <= exposures["ead"]]
64
65 collaterals = collaterals.dropna(subset=["client_id", "collateral_value"]).copy()
66 collaterals["client_id"] = collaterals["client_id"].astype(str).str.strip()
67 collaterals["collateral_value"] = pd.to_numeric(collaterals["collateral_value"], errors="coerce")
68 collaterals = collaterals.dropna(subset=["collateral_value"])
69 collaterals = collaterals[collaterals["collateral_value"] >= 0]
70
71
72
73 # 3) FUSION
74
75 df = exposures.merge(clients, on="client_id", how="left")
76
77 coll_agg = collaterals.groupby("client_id", as_index=False).agg(
78     total_collateral=("collateral_value", "sum"),
79     collateral_lines=("collateral_value", "size"),
80 )
81 df = df.merge(coll_agg, on="client_id", how="left")
82
83 df["total_collateral"] = df["total_collateral"].fillna(0.0)
84 df["collateral_lines"] = df["collateral_lines"].fillna(0).astype(int)
85
86
87
88 # 4) INDICATEURS
89
90 df["expected_loss"] = df["ead"] * df["pd"] * df["lgd"]
91 df["watchlist_flag"] = (df["pd"] > PD_WATCHLIST_THRESHOLD).astype(int)
92 df["collateral_coverage"] = np.where(df["ead"] > 0, df["total_collateral"] / df["ead"], 0.0)
93
94 # Basel III proxy
95 df["rwa"] = df["ead"] * RW_PROXY
96 df["cet1_capital"] = df["ead"] * CET1_RATE_PROXY
97 df["cet1_ratio"] = np.where(df["rwa"] > 0, df["cet1_capital"] / df["rwa"], 0.0)
98
99 df["cet1_ok_4_5"] = df["cet1_ratio"] >= BASEL_45
100 df["cet1_ok_8"] = df["cet1_ratio"] >= BASEL_8
101 df["cet1_ok_10_5"] = df["cet1_ratio"] >= BASEL_105

```

```

102
103 # 5) KPIs
104
105 kpi_country = df.groupby("country", dropna=False).agg(
106     n_clients=("client_id", "count"),
107     total_loan=("loan_amount", "sum"),
108     total_ead=("ead", "sum"),
109     avg_pd=("pd", "mean"),
110     total_el=("expected_loss", "sum"),
111     watchlist_rate=("watchlist_flag", "mean"),
112     avg_collateral_coverage=("collateral_coverage", "mean"),
113 ).reset_index().sort_values("total_ead", ascending=False)
114
115 kpi_sector = df.groupby("sector", dropna=False).agg(
116     n_clients=("client_id", "count"),
117     total_loan=("loan_amount", "sum"),
118     total_ead=("ead", "sum"),
119     avg_pd=("pd", "mean"),
120     total_el=("expected_loss", "sum"),
121     watchlist_rate=("watchlist_flag", "mean"),
122     avg_collateral_coverage=("collateral_coverage", "mean"),
123 ).reset_index().sort_values("total_ead", ascending=False)
124
125 base_summary = pd.DataFrame({
126     "threshold": ["CET1 ≥ 4.5%", "CET1 ≥ 8%", "CET1 ≥ 10.5%"],
127     "clients_compliant": [
128         int(df["cet1_ok_4_5"].sum()),
129         int(df["cet1_ok_8"].sum()),
130         int(df["cet1_ok_10_5"].sum())
131     ],
132     "compliance_rate_%": [
133         float(df["cet1_ok_4_5"].mean() * 100),
134         float(df["cet1_ok_8"].mean() * 100),
135         float(df["cet1_ok_10_5"].mean() * 100)
136     ]
137 })
138
139 ead_total = df["ead"].sum()
140 weighted_pd = (df["pd"] * df["ead"]).sum() / ead_total if ead_total > 0 else 0.0
141
142 summary = pd.DataFrame({
143     "metric": [
144         "Number of clients",
145         "Total loan amount",
146         "Total EAD",
147         "EAD-weighted PD",
148         "Total expected loss",
149         "WatchList rate",
150         "Average collateral coverage"
151     ],
152     "value": [
153         len(df),
154         df["loan_amount"].sum(),
155         ead_total,
156         weighted_pd,
157         df["expected_loss"].sum(),
158         df["watchlist_flag"].mean(),
159         df["collateral_coverage"].mean()
160     ]
161 })
162
163
164 # 6) GRAPHIQUES
165
166 FIG_DIR.mkdir(parents=True, exist_ok=True)
167
168 pd_plot = FIG_DIR / "pd_distribution.png"
169 ead_plot = FIG_DIR / "ead_by_country.png"
170 wl_plot = FIG_DIR / "watchlist_by_sector.png"
171
172 plt.figure()
173 sns.histplot(df["pd"], bins=40, kde=True)
174 plt.title("PD Distribution")
175 plt.tight_layout()
176 plt.savefig(pd_plot)
177 plt.close()
178
179 plt.figure()
180 sns.barplot(data=kpi_country.head(10), x="country", y="total_ead")
181 plt.title("Total EAD by Country (Top 10)")
182 plt.xticks(rotation=30, ha="right")
183 plt.tight_layout()
184 plt.savefig(ead_plot)
185 plt.close()

```

```

186
187 plt.figure()
188 sns.barplot(data=kpi_sector.head(10), x="sector", y="watchlist_rate")
189 plt.title("WatchList Rate by Sector (Top 10 by EAD)")
190 plt.xticks(rotation=30, ha="right")
191 plt.tight_layout()
192 plt.savefig(wl_plot)
193 plt.close()
194
195
196 # 7) OUTPUT
197
198 OUTPUT_EXCEL.parent.mkdir(parents=True, exist_ok=True)
199
200 with pd.ExcelWriter(OUTPUT_EXCEL, engine="openpyxl") as writer:
201     df.to_excel(writer, sheet_name="clients_detail", index=False)
202     kpi_country.to_excel(writer, sheet_name="kpi_country", index=False)
203     kpi_sector.to_excel(writer, sheet_name="kpi_sector", index=False)
204     basel_summary.to_excel(writer, sheet_name="basel_compliance", index=False)
205     summary.to_excel(writer, sheet_name="summary", index=False)
206
207     # Feuille "graphs"
208     wb = writer.book
209     ws = wb.create_sheet("graphs")
210
211     ws["A1"] = "PD Distribution"
212     ws["A25"] = "Total EAD by Country (Top 10)"
213     ws["A49"] = "WatchList Rate by Sector (Top 10 by EAD)"
214
215     img1 = XLImage(str(pd_plot))
216     img2 = XLImage(str(ead_plot))
217     img3 = XLImage(str(wl_plot))
218
219     # Positionnement des images
220     ws.add_image(img1, "A1")
221     ws.add_image(img2, "A35")
222     ws.add_image(img3, "A70")
223
224     print("Fichier Excel final créé :", OUTPUT_EXCEL)

```

IV. Résultat attendu

Le résultat final du projet est un **fichier Excel de reporting unique**, généré automatiquement par le script Python, contenant :

- une feuille avec les **données clients détaillées après traitement**,
- des tableaux de synthèse par pays et par secteur,
- un résumé global des indicateurs clés de risque,
- une feuille intégrant directement les graphiques produits.

Ce fichier est directement exploitable pour une analyse ou une présentation, sans manipulation manuelle supplémentaire.

V. Organisation du code et choix techniques

Le projet est structuré autour de **deux scripts Python distincts** :

1. Un script de **génération de données**, permettant de créer une base de travail en l'absence de données réelles.
2. Un script de **traitement et d'analyse**, organisé selon un enchaînement clair : lecture → nettoyage → jointures → transformations → visualisations → export.

Le code utilise exclusivement le langage **Python**, avec les bibliothèques standards vues en cours.

VI. Hypothèses et limites

Le script de traitement suppose un **schéma de données défini**, avec des noms de colonnes standardisés.

En cas de modification de la structure de la base ou des intitulés de colonnes, une **adaptation mineure du code serait nécessaire**, ce qui est cohérent avec les pratiques industrielles où les schémas de données sont documentés et stabilisés.

Les calculs réglementaires (Watchlist, ratios CET1) reposent sur des **hypothèses simplifiées à visée pédagogique** et ne constituent pas des calculs réglementaires réels.

Annexe

Code génération de données :

```
import numpy as np

import pandas as pd

from pathlib import Path


# PARAMETRES


N_CLIENTS = 10000


# Fichier Final

OUTPUT_FILE = Path("/Users/nathalyqiu/Desktop/data/raw/bank_database.xlsx")


# (optionnel)

SEED = 42


# GENERATION DES DONNEES


rng = np.random.default_rng(SEED)


countries = ["France", "Germany", "Italy", "Spain", "Belgium", "Netherlands"]

sectors = ["Industry", "Services", "Retail", "Energy", "Tech", "RealEstate"]

ratings = ["AAA", "AA", "A", "BBB", "BB", "B"]


# ----- TABLE CLIENTS -----
```

```

clients = pd.DataFrame()

clients["client_id"] = [f"C{i:05d}" for i in range(1, N_CLIENTS + 1)]

clients["country"] = rng.choice(countries, size=N_CLIENTS)

clients["sector"] = rng.choice(sectors, size=N_CLIENTS)

clients["rating"] = rng.choice(ratings, size=N_CLIENTS, p=[0.03, 0.07, 0.20, 0.35, 0.25, 0.10])


# ----- TABLE EXPOSURES -----

# EAD : distribution réaliste très étalée

ead = rng.lognormal(mean=12.0, sigma=0.8, size=N_CLIENTS)

ead = np.clip(ead, 10000, 15000000)


# Montant du prêt :

ratio_loan = rng.uniform(0.6, 1.0, size=N_CLIENTS)

loan_amount = ead * ratio_loan


# PD et LGD

pd_values = rng.beta(a=1.5, b=50, size=N_CLIENTS) * 0.25

pd_values = np.clip(pd_values, 0.0001, 0.20)


lgd_values = rng.beta(a=2.0, b=2.5, size=N_CLIENTS)

lgd_values = np.clip(lgd_values, 0.10, 0.90)


exposures = pd.DataFrame()

exposures["client_id"] = clients["client_id"]

exposures["loan_amount"] = loan_amount.round(2)

```

```

exposures["ead"] = ead.round(2)

exposures["pd"] = pd_values.round(6)

exposures["lgd"] = lgd_values.round(4)


# ----- TABLE COLLATERALS -----

n_lines = rng.integers(0, 4, size=N_CLIENTS) # 0,1,2,3
client_ids = np.repeat(clients["client_id"].values, n_lines)


collateral_value = rng.lognormal(mean=10.5, sigma=0.7, size=len(client_ids))
collateral_value = np.clip(collateral_value, 5000, 5000000)


collaterals = pd.DataFrame()

collaterals["client_id"] = client_ids

collaterals["collateral_value"] = collateral_value.round(2)


# EXPORT EXCEL

OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)

with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:

    clients.to_excel(writer, sheet_name="clients", index=False)

    exposures.to_excel(writer, sheet_name="exposures", index=False)

    collaterals.to_excel(writer, sheet_name="collaterals", index=False)


print("Fichier Excel généré :", OUTPUT_FILE)

```

```
print("Clients :", len(clients))  
print("Exposures :", len(exposures))  
print("Collaterals lines :", len(collaterals))
```

Code traitements de données :

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from pathlib import Path  
from openpyxl.drawing.image import Image as XLImage
```

```
# PARAMETRES
```

```
# Fichier source
```

```
INPUT_EXCEL = Path("/Users/nathalyqiu/Desktop/data/raw/bank_database.xlsx")
```

```
# Fichier final
```

```
OUTPUT_EXCEL = Path("/Users/nathalyqiu/Desktop/data/raw/bank_risk_report.xlsx")
```

```
PD_WATCHLIST_THRESHOLD = 0.03 # Seuil = Zone de vigilance utilisé par les banques
```

```

# Basel III (proxy)

BASEL_45 = 0.045 # Règlementation = CET1 min

BASEL_8 = 0.08 # Tier 1 + Tier 2 = Capital total min

BASEL_105 = 0.105 # CET1 + Buffer


RW_PROXY = 1.0 # Cas Standard

CET1_RATE_PROXY = 0.12 # Banque prudente donc sup à 10,5%


# Dossier pour images

FIG_DIR = Path("/Users/nathalyqiu/Desktop/data/raw/figures_tmp")


# 1) DONNÉES

if not INPUT_EXCEL.exists():

    raise FileNotFoundError(f"Fichier introuvable : {INPUT_EXCEL}")

clients = pd.read_excel(INPUT_EXCEL, sheet_name="clients")

exposures = pd.read_excel(INPUT_EXCEL, sheet_name="exposures")

collaterals = pd.read_excel(INPUT_EXCEL, sheet_name="collaterals")


# 2) NETTOYAGE

```

```

clients = clients.dropna(subset=["client_id"]).copy()

clients["client_id"] = clients["client_id"].astype(str).str.strip()

clients["country"] = clients["country"].astype(str).str.strip()

clients["sector"] = clients["sector"].astype(str).str.strip()

clients["rating"] = clients["rating"].astype(str).str.strip().str.upper()


exposures = exposures.dropna(subset=["client_id", "loan_amount", "ead", "pd", "lgd"]).copy()

exposures["client_id"] = exposures["client_id"].astype(str).str.strip()


exposures["loan_amount"] = pd.to_numeric(exposures["loan_amount"], errors="coerce")

exposures["ead"] = pd.to_numeric(exposures["ead"], errors="coerce")

exposures["pd"] = pd.to_numeric(exposures["pd"], errors="coerce")

exposures["lgd"] = pd.to_numeric(exposures["lgd"], errors="coerce")

exposures = exposures.dropna(subset=["loan_amount", "ead", "pd", "lgd"])


exposures = exposures[(exposures["loan_amount"] > 0) & (exposures["ead"] > 0)]

exposures = exposures[exposures["pd"].between(0, 1)]

exposures = exposures[exposures["lgd"].between(0, 1)]

exposures = exposures[exposures["loan_amount"] <= exposures["ead"]]


collaterals = collaterals.dropna(subset=["client_id", "collateral_value"]).copy()

collaterals["client_id"] = collaterals["client_id"].astype(str).str.strip()

collaterals["collateral_value"] = pd.to_numeric(collaterals["collateral_value"], errors="coerce")

collaterals = collaterals.dropna(subset=["collateral_value"])

collaterals = collaterals[collaterals["collateral_value"] >= 0]

```

3) FUSION

```
df = exposures.merge(clients, on="client_id", how="left")

coll_agg = collaterals.groupby("client_id", as_index=False).agg(
    total_collateral=("collateral_value", "sum"),
    collateral_lines=("collateral_value", "size"),
)

df = df.merge(coll_agg, on="client_id", how="left")

df["total_collateral"] = df["total_collateral"].fillna(0.0)
df["collateral_lines"] = df["collateral_lines"].fillna(0).astype(int)
```

4) INDICATEURS

```
df["expected_loss"] = df["ead"] * df["pd"] * df["lgd"]

df["watchlist_flag"] = (df["pd"] > PD_WATCHLIST_THRESHOLD).astype(int)

df["collateral_coverage"] = np.where(df["ead"] > 0, df["total_collateral"] / df["ead"], 0.0)

# Basel III proxy
```



```
df["rwa"] = df["ead"] * RW_PROXY
df["cet1_capital"] = df["ead"] * CET1_RATE_PROXY
df["cet1_ratio"] = np.where(df["rwa"] > 0, df["cet1_capital"] / df["rwa"], 0.0)
```

```
df["cet1_ok_4_5"] = df["cet1_ratio"] >= BASEL_45
df["cet1_ok_8"] = df["cet1_ratio"] >= BASEL_8
df["cet1_ok_10_5"] = df["cet1_ratio"] >= BASEL_105
```

5) KPIs

```
kpi_country = df.groupby("country", dropna=False).agg(
    n_clients=("client_id", "count"),
    total_loan=("loan_amount", "sum"),
    total_ead=("ead", "sum"),
    avg_pd=("pd", "mean"),
    total_el=("expected_loss", "sum"),
    watchlist_rate=("watchlist_flag", "mean"),
    avg_collateral_coverage=("collateral_coverage", "mean"),
).reset_index().sort_values("total_ead", ascending=False)
```

```
kpi_sector = df.groupby("sector", dropna=False).agg(
    n_clients=("client_id", "count"),
    total_loan=("loan_amount", "sum"),
    total_ead=("ead", "sum"),
    avg_pd=("pd", "mean"),
```

```

total_el=("expected_loss", "sum"),
watchlist_rate=("watchlist_flag", "mean"),
avg_collateral_coverage=("collateral_coverage", "mean"),
).reset_index().sort_values("total_ead", ascending=False)

```

```

basel_summary = pd.DataFrame({
    "threshold": ["CET1  $\geq$  4.5%", "CET1  $\geq$  8%", "CET1  $\geq$  10.5%"],
    "clients_compliant": [
        int(df["cet1_ok_4_5"].sum()),
        int(df["cet1_ok_8"].sum()),
        int(df["cet1_ok_10_5"].sum())
    ],
    "compliance_rate_%": [
        float(df["cet1_ok_4_5"].mean() * 100),
        float(df["cet1_ok_8"].mean() * 100),
        float(df["cet1_ok_10_5"].mean() * 100)
    ]
})

```

```

ead_total = df["ead"].sum()

```

```

weighted_pd = (df["pd"] * df["ead"]).sum() / ead_total if ead_total > 0 else 0.0

```

```

summary = pd.DataFrame({
    "metric": [
        "Number of clients",

```

```

    "Total loan amount",
    "Total EAD",
    "EAD-weighted PD",
    "Total expected loss",
    "WatchList rate",
    "Average collateral coverage"
],
"value": [
    len(df),
    df["loan_amount"].sum(),
    ead_total,
    weighted_pd,
    df["expected_loss"].sum(),
    df["watchlist_flag"].mean(),
    df["collateral_coverage"].mean()
]
})

```

6) GRAPHIQUES

```
FIG_DIR.mkdir(parents=True, exist_ok=True)
```

```
pd_plot = FIG_DIR / "pd_distribution.png"
```

```
ead_plot = FIG_DIR / "ead_by_country.png"
```

```
wl_plot = FIG_DIR / "watchlist_by_sector.png"
```

```
plt.figure()
```

```
sns.histplot(df["pd"], bins=40, kde=True)
```

```
plt.title("PD Distribution")
```

```
plt.tight_layout()
```

```
plt.savefig(pd_plot)
```

```
plt.close()
```

```
plt.figure()
```

```
sns.barplot(data=kpi_country.head(10), x="country", y="total_ead")
```

```
plt.title("Total EAD by Country (Top 10)")
```

```
plt.xticks(rotation=30, ha="right")
```

```
plt.tight_layout()
```

```
plt.savefig(ead_plot)
```

```
plt.close()
```

```
plt.figure()
```

```
sns.barplot(data=kpi_sector.head(10), x="sector", y="watchlist_rate")
```

```
plt.title("WatchList Rate by Sector (Top 10 by EAD)")
```

```
plt.xticks(rotation=30, ha="right")
```

```
plt.tight_layout()
```

```
plt.savefig(wl_plot)
```

```
plt.close()
```

7) OUTPUT

```
OUTPUT_EXCEL.parent.mkdir(parents=True, exist_ok=True)
```

```
with pd.ExcelWriter(OUTPUT_EXCEL, engine="openpyxl") as writer:
```

```
    df.to_excel(writer, sheet_name="clients_detail", index=False)
```

```
    kpi_country.to_excel(writer, sheet_name="kpi_country", index=False)
```

```
    kpi_sector.to_excel(writer, sheet_name="kpi_sector", index=False)
```

```
    basel_summary.to_excel(writer, sheet_name="basel_compliance", index=False)
```

```
    summary.to_excel(writer, sheet_name="summary", index=False)
```

```
# Feuille "graphs"
```

```
wb = writer.book
```

```
ws = wb.create_sheet("graphs")
```

```
ws["A1"] = "PD Distribution"
```

```
ws["A25"] = "Total EAD by Country (Top 10)"
```

```
ws["A49"] = "WatchList Rate by Sector (Top 10 by EAD)"
```

```
img1 = XLImage(str(pd_plot))
```

```
img2 = XLImage(str(ead_plot))
```

```
img3 = XLImage(str(wl_plot))
```

```
# Positionnement des images
```

```
ws.add_image(img1, "A1")
```

```
ws.add_image(img2, "A35")
```

```
ws.add_image(img3, "A70")
```

```
print("Fichier Excel final créé :", OUTPUT_EXCEL)
```